

LAB 4 — Weather Stations Monitoring

Net-Centric Computing | Fall 2025-2026

Kafka + Java

+ Docker (required)

+ Kubernetes (required)

1 Objective

Build a **distributed IoT stream-processing system** that simulates 10 weather stations emitting live readings into **Apache Kafka**, processes the stream in real-time to detect weather conditions, and persists all readings to a **relational SQL database**. The entire system is containerised with **Docker** and orchestrated with **Kubernetes**.

By the end you will:

- Understand producer/consumer messaging patterns through Kafka.
- Implement stream filtering and event detection with Kafka Streams / Processors.
- Persist high-throughput data efficiently using batch SQL inserts.
- Write Dockerfiles for multiple cooperating services.
- Deploy a multi-service system on Kubernetes with persistent storage.
- [Bonus] Deploy the full system to a real cloud across two separate VMs.

2 Background

System in one sentence: 10 weather station processes produce JSON messages every second to Kafka; a central station consumes them, detects rain, and stores everything in PostgreSQL.

Why Kafka fits this architecture:

IoT Challenge	Kafka Solution
10 stations emit 1 msg/sec each — 10 msg/sec total	Kafka topics decouple producers from consumers; handles millions of msgs/sec with ease.
Consumer may lag behind producers	Kafka retains messages on disk — the central station can replay from any offset.
Detect raining condition (humidity > 70%) in real-time	Kafka Streams Processor API filters the stream and publishes alerts to a separate topic.
Persist all readings without hammering the DB	Central station buffers records and does batch INSERT every 5,000 rows.

Message Flow
WeatherStation[1..10]
-- JSON msg every 1s --> Kafka topic: weather-readings
KafkaProcessor
-- consumes weather-readings
-- humidity > 70% ? --> Kafka topic: rain-alerts
CentralStation
-- consumes weather-readings --> batch INSERT to SQL DB
-- consumes rain-alerts --> log / alert (optional)

3 Prerequisites & Cluster Setup

Component	Version / Notes
Docker	Latest stable. Install from https://www.docker.com/products/docker-desktop
Kubernetes	Minikube, Kind, or any cloud-managed cluster (GKE / EKS / AKS).
Java	17+ (for Kafka client libraries and station code)
Kafka	Bitnami Kafka image (includes Zookeeper). No local install needed — runs in K8s.
PostgreSQL	Run as a Kubernetes pod with a PersistentVolume, or use a managed provider.

Quick smoke-test (local Docker, before K8s):

```
docker run -d --name kafka -p 9092:9092 \
  -e KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092 \
  bitnami/kafka:latest
docker exec -it kafka kafka-topics.sh --create --topic weather-readings \
  --bootstrap-server localhost:9092 --partitions 3 --replication-factor 1
```

4 Implementation

A) Weather Station Mock (Java)

Each of the 10 station processes emits one message per second. Implement the distribution and drop logic below, then serialize to JSON and send to Kafka.

Field	Rule	Detail
battery_status	Random weighted	Low 30% / Medium 40% / High 30%
s_no	Auto-increment	Per-station counter, starts at 1
Drop	Random 10%	Silently skip send — do NOT increment s_no
humidity	Random int	0–100 (%)
temperature	Random int	Fahrenheit range
wind_speed	Random int	km/h range

Weather Status Message Schema		
{		
"station_id":	1,	// Long
"s_no":	1,	// Long – auto-incremental per station
"battery_status":	"low",	// "low" "medium" "high"
"status_timestamp":	1681521224,	// Long Unix timestamp
"weather":	{	
"humidity":	35,	// Integer %
"temperature":	100,	// Integer Fahrenheit
"wind_speed":	13	// Integer km/h
	}	
}		

Kafka Producer skeleton (Java)
Properties props = new Properties();
props.setProperty(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka-svc:9092");
props.setProperty(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
props.setProperty(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
try (KafkaProducer<String,String> producer = new KafkaProducer<>(props)) {
while (true) {
if (Math.random() >= 0.10) { // 10% drop
String json = buildMessage(stationId, ++seqNo);
producer.send(new ProducerRecord<>("weather-readings",
String.valueOf(stationId), json));
}
Thread.sleep(1000);
}
}

B) Kafka Stream Processor — Rain Detection

Use the Kafka Streams `Processor` API (or the high-level DSL) to consume `weather-readings`, check **humidity > 70%**, and forward matching messages to the `rain-alerts` topic.

KafkaStreams DSL sketch
StreamsBuilder builder = new StreamsBuilder();
builder.stream("weather-readings",
Consumed.with(Serdes.String(), Serdes.String()))
.filter((k, v) -> parseHumidity(v) > 70)
.to("rain-alerts", Produced.with(Serdes.String(), Serdes.String()));
KafkaStreams streams = new KafkaStreams(builder.build(), streamsConfig);
streams.start();

C) Central Station — Consumer & DB Persistence

The Central Station consumes `weather-readings` and writes them to PostgreSQL. Use **batch inserts of 5,000 records** to avoid excessive I/O on the database.

PostgreSQL Schema (minimum)	
CREATE TABLE weather_readings (	
id	BIGSERIAL PRIMARY KEY,
station_id	BIGINT,
sequence_number	BIGINT,
battery_status	VARCHAR(10),
timestamp	BIGINT,
humidity	INT,
temperature	INT,
wind_speed	INT
);	

D) SQL Analysis Queries

Include both queries in your report output — run them after letting the system run for at least 5 minutes.

Query 1 — Battery distribution per station	
SELECT station_id,	
battery_status,	
COUNT(*)	AS total,
ROUND(100.0 * COUNT(*) /	
SUM(COUNT(*)) OVER (PARTITION BY station_id), 1) AS pct	
FROM weather_readings	
GROUP BY station_id, battery_status	
ORDER BY station_id, battery_status;	
-- Expected: ~30% low / 40% medium / 30% high per station	

Query 2 — Dropped messages per station	
SELECT station_id,	
MAX(sequence_number)	AS highest_seq,
COUNT(*)	AS received,
MAX(sequence_number) - COUNT(*)	AS estimated_dropped
FROM weather_readings	
GROUP BY station_id	
ORDER BY station_id;	
-- estimated_dropped should be ~10% of highest_seq	

5 Kubernetes Deployment (Required)

All services must run as Kubernetes pods. The minimum set of YAML files is listed below.

YAML File / Object	What it defines
kafka-deployment.yaml	Kafka + Zookeeper as a single pod (bitnami/kafka). Exposed via ClusterIP Service on port 9092.
weather-station-deployment.yaml	Deployment with replicas: 10. Each pod runs your WeatherStation JAR; STATION_ID injected via env var using the downward API.
central-station-deployment.yaml	Single pod running the consumer + batch-insert logic.
postgres-deployment.yaml	PostgreSQL pod backed by a PersistentVolumeClaim (at least 1Gi).
postgres-pvc.yaml	PersistentVolumeClaim for the database. Data survives pod restarts.
configmap.yaml	Kafka bootstrap address, DB connection string (non-secret values).
secret.yaml	DB username and password — base64 encoded. Never hardcode in source.

Key commands:

```
kubectl apply -f k8s/           # apply all YAMLs at once
kubectl get pods -w             # watch all 13 pods come up
kubectl logs -f <central-pod>   # tail the consumer logs
kubectl exec -it <postgres-pod> -- psql -U weather_user weather_db
```

Mini-Task — Verify end-to-end (answer in report)

1. After 2 minutes, run Query 1 (battery distribution). Do the percentages match the spec?
2. Run Query 2 (dropped messages). Is the drop rate close to 10%?
3. Scale weather stations to 5: `kubectl scale deployment weather-station --replicas=5`
-- Does the DB row count per second drop proportionally? Why?

6 Bonus — Cloud Deployment

Deploy the system on **two separate cloud VMs** (no Kubernetes). Stations and the central station must be on different machines.

VM	Hosts
VM 1 — Weather Stations	All 10 station processes (Docker containers or plain Java). Must connect to Kafka on VM 2 over the internet.
VM 2 — Central	Kafka + Zookeeper, Central Station service, SQL database (or use a managed DB).

Bonus B — Managed : Deploy PostgreSQL on Aiven Cloud (free tier). Configure your Central Station to connect over TLS. Do not hardcode credentials in source code.

Evidence required in report:

- Cloud provider, VM specs, OS, open ports / security groups.
- Screenshot: station logs showing messages sent.
- Screenshot: central station logs showing rows inserted.
- Screenshot: SQL query results after 5 minutes of runtime.

7 Deliverables & Grading

Report must include: system architecture diagram (hand-drawn is fine), implementation walkthrough of each component, SQL query outputs with commentary, and one paragraph connecting the design to concepts from van Steen & Tanenbaum Ch. 4 (communication), Ch. 6 (coordination), and Ch. 8 (fault tolerance).

Policies & Resources

- Groups of 2–3. Every member must understand the full solution.
- No late submissions without prior written approval.
- Code sharing between groups = plagiarism.

Key resources:

- Apache Kafka docs: <https://kafka.apache.org/documentation/>
- Kafka Streams API: <https://kafka.apache.org/documentation/streams/>
- Bitnami Kafka image: <https://hub.docker.com/r/bitnami/kafka>
- Kubernetes Deployments: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- Aiven Cloud (managed DB): <https://aiven.io>
- Textbook: van Steen & Tanenbaum — Distributed Systems, 3rd Ed. Ch. 4, 6, 8